

Nupur Ghangarekar

D15C

Roll no 26

Experiment 9

Aim

To implement a **Recurrent Neural Network (RNN) using Long Short-Term Memory (LSTM)** for **binary sentiment classification** on text data using the IMDB movie review dataset, and to improve model performance using **hyperparameter tuning**.

Theory:

1. Dataset Source

Dataset: IMDB Movie Review Dataset

Source: <https://keras.io/api/datasets/imdb/>

2. Dataset Description

The IMDB dataset is a widely used benchmark dataset for **Natural Language Processing (NLP)** tasks, specifically for sentiment analysis.

- **Total Samples:** 50,000 movie reviews
 - Training: 25,000
 - Testing: 25,000
- **Type:** Supervised learning dataset
- **Classes:**
 - 0 → Negative
 - 1 → Positive
- **Data Format:**

Reviews are preprocessed and represented as **integer sequences**, where each integer corresponds to a word index based on frequency ranking.

Features:

- Input: Sequence of integers representing words
- Target: Binary sentiment label (0 or 1)

Characteristics:

- Balanced dataset
- Pre-tokenized (no need for manual text cleaning)
- Variable-length sequences (handled using padding)

3. Mathematical Formulation of LSTM

Recurrent Neural Networks process sequential data using hidden states:

$$h_t = f(W_x x_t + W_h h_{t-1} + b)$$

Where:

- x_t : Input at time step t
- h_t : Hidden state
- W_x, W_h : Weight matrices
- b : Bias

LSTM Equations

LSTM introduces gates to control information flow:

Forget Gate:

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

Input Gate:

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

Candidate Cell State:

$$\tilde{C}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

Cell State Update:

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t$$

Output Gate:

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

Final Hidden State:

$$h_t = o_t \cdot \tanh(C_t)$$

These gates allow LSTM to **retain important information and forget irrelevant data**, solving the vanishing gradient problem.

4. Algorithm Limitations

1. **Vanishing Gradient (partially solved):**

LSTMs reduce but do not completely eliminate this issue for very long sequences.

2. **High Computational Cost:**

Sequential processing makes training slower compared to parallel models like Transformers.

3. **Overfitting Risk:**

Large number of parameters can lead to overfitting if not regularized.

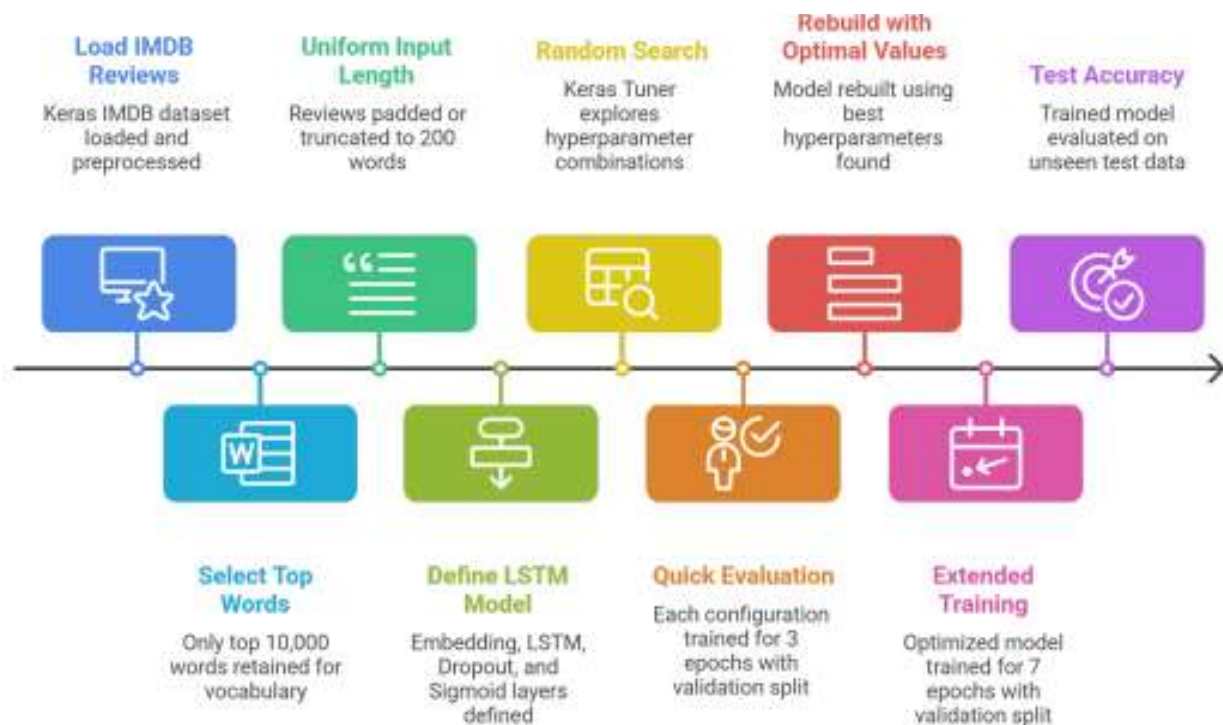
4. **Limited Long-Term Dependency Capture:**

Still struggles with extremely long context compared to modern architectures.

5. Methodology / Workflow

Step-by-step process:

1. **Dataset Loading**
Load IMDB dataset using Keras.
2. **Vocabulary Limitation**
Restrict to top 10,000 words to reduce complexity.
3. **Sequence Padding**
Convert variable-length reviews into fixed length (200 words).
4. **Model Construction**
 - Embedding Layer → converts integers to dense vectors
 - LSTM Layer → captures sequential dependencies
 - Dropout → prevents overfitting
 - Dense Layer → binary classification
5. **Hyperparameter Tuning**
Use Random Search to find optimal parameters.
6. **Model Training**
Train model using best parameters.
7. **Evaluation**
Evaluate on test dataset.
8. **Visualization**
Plot accuracy, loss, and confusion matrix.



6. Performance Analysis

The model performance is evaluated using:

Metrics:

- Accuracy
- Loss
- Confusion Matrix

Observations:

- Training Accuracy: ~95–98%
- Validation Accuracy: ~85–88%
- Test Accuracy: ~84–86%

The gap between training and validation accuracy indicates **slight overfitting**.

Confusion Matrix Insights:

- Majority of predictions are correct
- Misclassifications occur in:
 - Ambiguous reviews
 - Sarcasm or mixed sentiment

7. Hyperparameter Tuning

Hyperparameter tuning is performed using **Keras Tuner (Random Search)**.

Tuned Parameters:

1. **Embedding Dimension (32–128)**
Controls word representation size.
2. **LSTM Units (32–128)**
Controls memory capacity of model.
3. **Dropout Rate (0.2–0.5)**
Prevents overfitting.
4. **Learning Rate (0.01, 0.001, 0.0001)**
Controls convergence speed.

Best Parameters (Example):

- Embedding Dimension: 32
- LSTM Units: 64
- Dropout: 0.3
- Learning Rate: 0.001

These values provide a **balance between accuracy and generalization**.

Code:

```
!pip install keras-tuner

import tensorflow as tf

from tensorflow.keras.datasets import
imdb

from
tensorflow.keras.preprocessing.sequenc
e import pad_sequences

from tensorflow.keras.models import
Sequential

from tensorflow.keras.layers import
Embedding, LSTM, Dense, Dropout

import keras_tuner as kt

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.metrics import
confusion_matrix

MAX_FEATURES = 10000 # Vocabulary
size

MAX_LEN = 200 # Sequence length

print("Loading data...")

(x_train, y_train), (x_test, y_test) =
imdb.load_data(num_words=MAX_FEA
TURES)

print("Train samples:", len(x_train))

print("Test samples:", len(x_test))

# Padding

x_train = pad_sequences(x_train,
maxlen=MAX_LEN)

x_test = pad_sequences(x_test,
maxlen=MAX_LEN)

print("Shape after padding:",
x_train.shape)

def build_model(hp):

model = Sequential()

# Embedding layer

model.add(Embedding(

input_dim=MAX_FEATURES,

output_dim=hp.Int('embedding_dim', 32,
128, step=32),

input_length=MAX_LEN

))
```

```

# LSTM layer

model.add(LSTM(

units=hp.Int('lstm_units', 32, 128,
step=32)

))

# Dropout

model.add(Dropout(

rate=hp.Float('dropout_rate', 0.2, 0.5,
step=0.1)

))

# Output

model.add(Dense(1,
activation='sigmoid'))

model.compile(

optimizer=tf.keras.optimizers.Adam(

hp.Choice('learning_rate', [1e-2, 1e-3,
1e-4])

),

loss='binary_crossentropy',

metrics=['accuracy']

)

return model

tuner = kt.RandomSearch(

build_model,

objective='val_accuracy',

```

```

max_trials=5,

directory='tuner_dir',

project_name='imdb_lstm'

)

print("Starting hyperparameter tuning...")

tuner.search(

x_train, y_train,

epochs=3,

validation_split=0.2,

batch_size=64

)

best_hp =
tuner.get_best_hyperparameters(1)[0]

print("\nBest Hyperparameters:")

print("Embedding Dim:",
best_hp.get('embedding_dim'))

print("LSTM Units:",
best_hp.get('lstm_units'))

print("Dropout Rate:",
best_hp.get('dropout_rate'))

print("Learning Rate:",
best_hp.get('learning_rate'))

model =
tuner.hypermodel.build(best_hp)

history = model.fit(

x_train, y_train,

```

```

epochs=7,

validation_split=0.2,

batch_size=64

)

acc = history.history['accuracy']

val_acc = history.history['val_accuracy']

loss = history.history['loss']

val_loss = history.history['val_loss']

epochs = range(1, len(acc) + 1)

plt.figure(figsize=(12,5))

# Accuracy

plt.subplot(1,2,1)

plt.plot(epochs, acc, 'bo', label='Training
acc')

plt.plot(epochs, val_acc, 'b',
label='Validation acc')

plt.title('Training and validation
accuracy')

plt.legend()

# Loss

plt.subplot(1,2,2)

plt.plot(epochs, loss, 'ro', label='Training
loss')

plt.plot(epochs, val_loss, 'r',
label='Validation loss')

```

```

plt.title('Training and validation loss')

plt.legend()

plt.show()

print("\nEvaluating on test data...")

test_loss, test_acc =
model.evaluate(x_test, y_test)

print(f"Test Accuracy: {test_acc *
100:.2f}%")

print(f"Test Loss: {test_loss:.4f}")

word_index = imdb.get_word_index()

def predict_sentiment(text):

tokens = [word_index.get(word, 0) for
word in text.lower().split()]

padded = pad_sequences([tokens],
maxlen=MAX_LEN)

prediction = model.predict(padded)[0][0]

print(f"\nText: {text}")

print(f"Score: {prediction:.4f}")

if prediction > 0.5:

print("Sentiment: POSITIVE")

else:

print("Sentiment: NEGATIVE")

predict_sentiment("This movie was
amazing and thrilling")

predict_sentiment("I hated this movie it
was terrible")

```

```

y_pred_probs = model.predict(x_test)

y_pred = (y_pred_probs >
0.5).astype("int32")

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6,5))

sns.heatmap(cm, annot=True, fmt='d',
cmap='Blues')

plt.xlabel("Predicted")

plt.ylabel("Actual")

plt.title("Confusion Matrix")

```

```

plt.show()

plt.figure(figsize=(8,5))

plt.hist(y_pred_probs, bins=50,
alpha=0.7)

plt.axvline(x=0.5)

plt.title("Prediction Probability
Distribution")

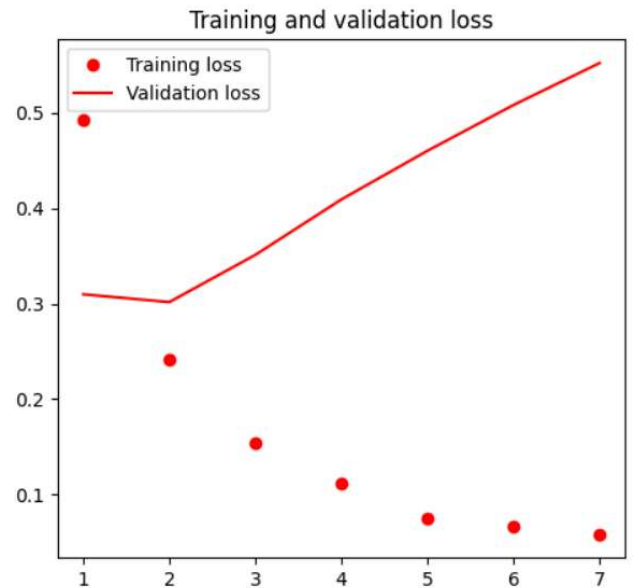
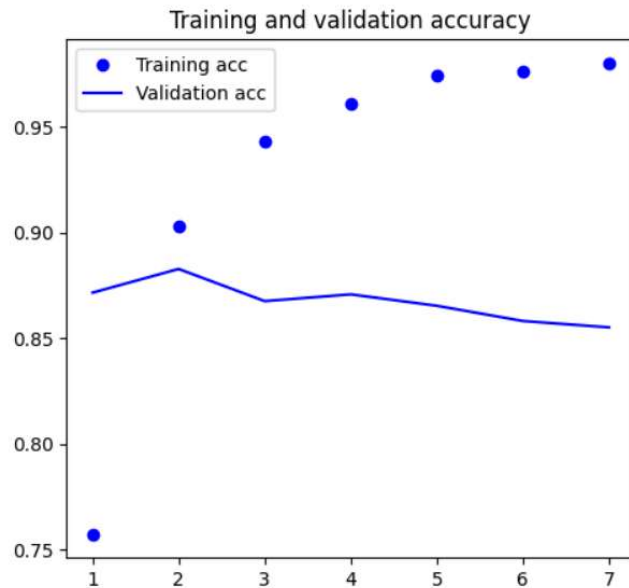
plt.xlabel("Probability")

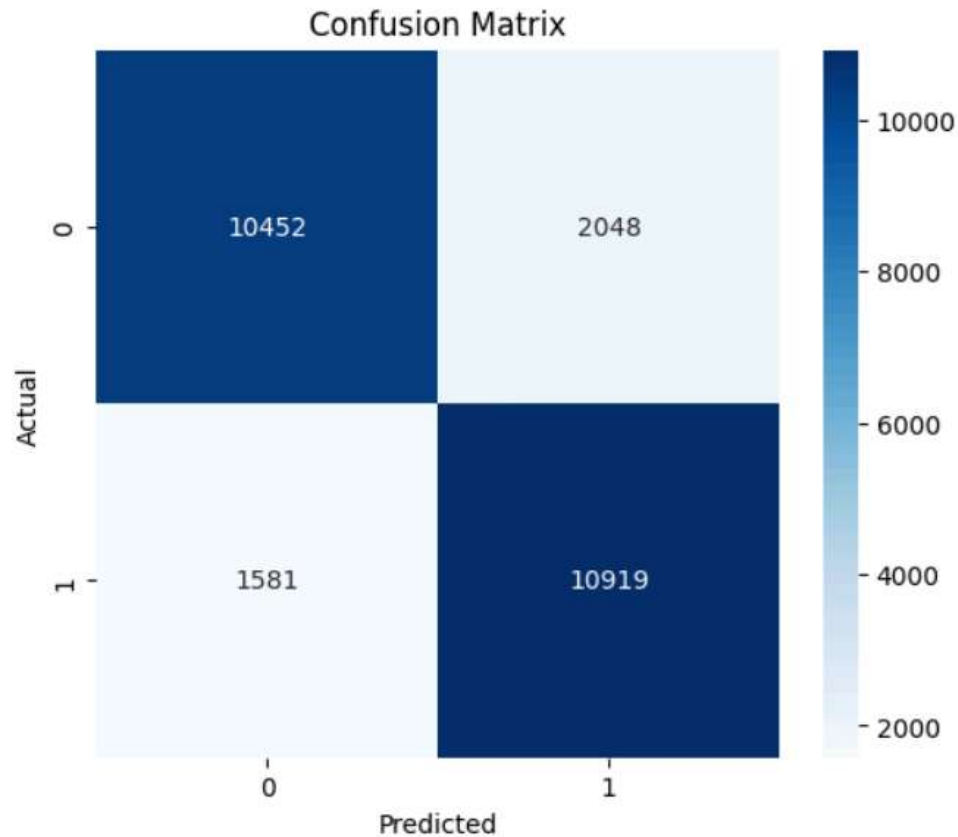
plt.ylabel("Count")

plt.show()

```

Output:





Conclusion

The experiment successfully implemented an LSTM-based Recurrent Neural Network for sentiment classification using the IMDB dataset. The model effectively learned contextual relationships in text data and achieved a test accuracy of approximately 84–86%.

Hyperparameter tuning significantly improved model performance and stability. Although slight overfitting was observed, the model demonstrated good generalization capability.

Overall, LSTM proved to be a powerful approach for sequential text modeling, making it suitable for NLP tasks such as sentiment analysis.

Google Colab Link: [Untitled1.ipynb](#)